M4.B4: Module 4 Priority Queues and Heaps Application

Programming Assignment

Author: Guan-De Liou

M4.B4: Module 4 Priority Queues and Heaps Application Programming Assignment

This Programming Assignment is based on Application Exercise 5.7.29:

- Describe an algorithm for the New Unknown Airline (NUA) Upgrade System.
- Develop a program that processes the request and cancellations for upgrade and provides the list of k-highest priority flyers among the *n* frequent flyers on the waiting list.
- Your implementation must process the request and cancellations in O(logn) time and find the k-highest-priority flyers in O(k logn) times using the data structures in Chapter 5.

In your submission, you must upload **two** files:

- Submit a typed Word or PDF document with description of your solution on LUMINA.
    - Your answers should be very clear, in proper order, and use complete sentences.
    - Review your work several times before submission to be sure the steps of the algorithm are clearly and properly stated and in the correct order.
    - Provide pseudocode for the main algorithms, except for user interface, input/output, etc.
- Submit a single zip file named NUA_Upgrade_System_Lastname.zip containing the code file and test files.

Here are some further guidelines for programming code:

- Use an OOP language, such as Java, Python or C++.
- Comment your code.
- Your code file must compile and accept any number of inputs in the format you specified.

Guidelines for Input/Output:

Below are the Input and Output formats. In addition, few test cases are included and your code should run against those. Such standard format

should be followed for other programming assignments as well!

Input:

n = Number of flyers in waiting list
k = Required K highest priority flyers
c = Number of cancellation requests

Now n lines followed with flyers information like, flyer_name, flyer _id, the time that flyer came into waiting list (request time) and the Status given to that flyer. So priority will be calculated as the combination of time and Status. The priority is determined FIRST by status, then request time in the waiting list, if there are tie in status, request times break the ties

For the simplicity time will be an integer ranging from 1 to 100000. And Status will be any from the following list [Super, Platinum, Gold, Silver], where Super > Platinum > Gold > Silver.

Now 'c' lines followed with flyer's id who raised the cancellation request.

Example Input-1
---
5 2 1 (n k c)

A 1 120 Platinum (flyer_name, flyer_id, time, status)
B 5 60 Super
C 3 140 Super
D 2 130 Gold
E 4 150 Silver

5 (flyer id who raised the cancellation request)

Example Input-2
---
4 1 2 (n k c)

A 1 30 Platinum (flyer_name, flyer_id, time, status)
B 2 60 Gold

C 3 90 Silver
D 4 120 Super

2 (flyer_id who raised the cancellation request)
4 (flyer_id who raised the cancellation request)

Output:

Output should be top K flyers based on priority, who are allowed to onboard.

Example Output-1

(C, 3, 140, Super)
(A, 1, 120, Platinum)

◎ 5.7.29 Question

EXERCISE | 5.7.29

(a) Suppose you work for a major airline and are given the job of writing the algorithm for processing upgrades into first class on various flights. Any frequent flyer can request an upgrade for his or her up-coming flight using this online system. Frequent flyers have different priorities, which are determined first by frequent flyer status (which can be, in order, silver, gold, platinum, and super) and then, if there are ties, by length of time in the waiting list. In addition, at any time prior to the flight, a frequent flyer can cancel his or her upgrade request (for instance, if he or she wants to take a different flight), using a confirmation code they got when he or she made his or her upgrade request. When it is time to determine upgrades for a flight that is about to depart, the gate agents inform the system of the number, k, of seats available in first class, and it needs to match those seats with the k highest-priority passengers on the waiting list. Describe a system that can process upgrade requests and cancellations in $O(k \log n)$ time and can determine the k highest-priority flyers on the waiting list in $O(k \log n)$ time, where n is the number of frequent flyers on the waiting list.

Feedback?

[5.7.29 Solution]

Part 1: Detailed Reasoning and Proof

1. Problem Understanding

Problem Analysis

We need to design a system for airline upgrade requests that:

1. Supports **upgrade requests** (adding a flyer to a priority list).
2. Supports **cancellation requests** (removing a flyer from the list by ID).
3. Can **extract the top k highest-priority flyers** from the list.
4. All operations must run in **O(k log n) time,** where n is the number of flyers on the list.

Priority Definition

A flyer's priority is determined by:

1. **Status:**
   Super > Platinum > Gold > Silver
2. **Request Time:**
   If statuses are equal, the flyer who requested earlier (smaller time value) has higher priority.

Key Data Structure Choice

To meet the time complexity requirements, we use a **max-heap (priority queue)** to maintain the flyers.

A heap allows us to:

- Insert a new flyer in **O(log n)** time.
- Remove an arbitrary flyer in **O(log n)** time (with the help of a hash map for O(1) lookups).
- Extract the maximum element (highest priority) in **O(log n)** time.

By extracting the maximum element k times, we get the top k flyers in **O(k log n)** time.

2. Solution Designing: Algorithm Description for the New Unknown Airline (NUA) Upgrade System

Step 1: Data Structures

• **Max-Heap (Priority Queue)**:
Stores Flyer objects, ordered by priority (status first, then time).

• **Hash Map (Dictionary)**:
Maps flyer_id to the corresponding Flyer object.
This allows O(1) time to find a flyer for cancellation.

Step 2: Core Operations

A. Upgrade Request (Insert)

1. Create a new Flyer object with the given name, id, time, and status.
2. Insert the Flyer into the max-heap. This operation takes **O(log n)** time.
3. Add the Flyer to the hash map with flyer_id as the key. This operation takes **O(1)** time.

B. Cancellation Request (Delete)

1. Use the hash map to find the Flyer object associated with the given flyer_id. This takes **O(1)** time.
2. Remove the Flyer from the max-heap. This operation takes **O(log n)** time.
3. Remove the entry from the hash map. This takes **O(1)** time.

C. Extract Top k Flyers

1. Initialize an empty list to store the results.
2. Perform the following k times:

- Extract the maximum element (highest priority flyer) from the heap. This takes **O(log n)** time.

- Add this flyer to the results list.
- Re-insert the flyer back into the heap to maintain the original list for future operations. This takes **O(log n)** time.

The total time for this operation is **O(k log n)**.

3. Time Complexity Proof

• **Insert/Delete**

Each heap operation (sift up/down) takes **O(log n)** time.

The hash map operations are **O(1)**.

*Thus, each request is **O(log n)***.

• **Extract Top k**

We perform k extract-max and insert operations, each **O(log n)**.

*Thus, total time is **O(k log n)***.

Part 2: Code Implementation (C language)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define MAX_NAME 100
#define MAX_FLYERS 100000

// Status ranking: Silver=1, Gold=2, Platinum=3, Super=4
typedef enum {
    SILVER = 1,
    GOLD = 2,
    PLATINUM = 3,
    SUPER = 4
} Status;

// Flyer structure
typedef struct {
    char name[MAX_NAME];
    int flyer_id;
    int request_time;
    Status status;
    bool is_cancelled;
} Flyer;

// Max Heap for priority queue
typedef struct {
    Flyer* flyers[MAX_FLYERS];
    int size;
    int* id_to_index; // Hash map: flyer_id -> heap index
} PriorityQueue;

// Function prototypes
PriorityQueue* createPriorityQueue();
void destroyPriorityQueue(PriorityQueue* pq);
Status stringToStatus(const char* status_str);
```

```
int comparePriority(Flyer* f1, Flyer* f2);
void swap(Flyer** a, Flyer** b, int* index_a, int* index_b, int* id_to_index);
void heapifyUp(PriorityQueue* pq, int index);
void heapifyDown(PriorityQueue* pq, int index);
void insertFlyer(PriorityQueue* pq, Flyer* flyer);
Flyer* extractMax(PriorityQueue* pq);
void cancelRequest(PriorityQueue* pq, int flyer_id);
void findTopKFlyers(PriorityQueue* pq, int k, Flyer** result, int* result_size);

// Create priority queue
PriorityQueue* createPriorityQueue() {
    PriorityQueue* pq = (PriorityQueue*)malloc(sizeof(PriorityQueue));
    pq->size = 0;
    pq->id_to_index = (int*)calloc(MAX_FLYERS, sizeof(int));
    for (int i = 0; i < MAX_FLYERS; i++) {
        pq->id_to_index[i] = -1; // -1 means not in heap
    }
    return pq;
}

// Destroy priority queue
void destroyPriorityQueue(PriorityQueue* pq) {
    for (int i = 0; i < pq->size; i++) {
        free(pq->flyers[i]);
    }
    free(pq->id_to_index);
    free(pq);
}

// Convert status string to enum
Status stringToStatus(const char* status_str) {
    if (strcmp(status_str, "Super") == 0) return SUPER;
    if (strcmp(status_str, "Platinum") == 0) return PLATINUM;
    if (strcmp(status_str, "Gold") == 0) return GOLD;
    return SILVER;
}

// Compare priority: return 1 if f1 has higher priority, -1 if f2, 0 if equal
```

```
int comparePriority(Flyer* f1, Flyer* f2) {
    if (f1->status != f2->status) {
        return (f1->status > f2->status) ? 1 : -1;
    }
    // Same status, earlier time wins (lower time value = higher priority)
    if (f1->request_time != f2->request_time) {
        return (f1->request_time < f2->request_time) ? 1 : -1;
    }
    return 0;
}

// Swap two flyers and update index mapping
void swap(Flyer** a, Flyer** b, int* index_a, int* index_b, int* id_to_index) {
    Flyer* temp = *a;
    *a = *b;
    *b = temp;

    // Update index mapping
    id_to_index[(*a)->flyer_id] = *index_a;
    id_to_index[(*b)->flyer_id] = *index_b;
}

// Heapify up (for insertion)
void heapifyUp(PriorityQueue* pq, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (comparePriority(pq->flyers[index], pq->flyers[parent]) > 0) {
            swap(&pq->flyers[index], &pq->flyers[parent],
                &index, &parent, pq->id_to_index);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify down (for extraction)
void heapifyDown(PriorityQueue* pq, int index) {
```

```
    while (true) {
        int largest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < pq->size && comparePriority(pq->flyers[left], pq->flyers[largest]) >
0) {
            largest = left;
        }
        if (right < pq->size && comparePriority(pq->flyers[right], pq-
>flyers[largest]) > 0) {
            largest = right;
        }

        if (largest != index) {
            swap(&pq->flyers[index], &pq->flyers[largest],
                &index, &largest, pq->id_to_index);
            index = largest;
        } else {
            break;
        }
    }
}
```

// Insert flyer into priority queue - O(log n)

```
void insertFlyer(PriorityQueue* pq, Flyer* flyer) {
    pq->flyers[pq->size] = flyer;
    pq->id_to_index[flyer->flyer_id] = pq->size;
    heapifyUp(pq, pq->size);
    pq->size++;
}
```

// Extract maximum priority flyer - O(log n)

```
Flyer* extractMax(PriorityQueue* pq) {
    if (pq->size == 0) return NULL;

    Flyer* max = pq->flyers[0];
    pq->id_to_index[max->flyer_id] = -1;
```

```
    pq->size--;
    if (pq->size > 0) {
        pq->flyers[0] = pq->flyers[pq->size];
        pq->id_to_index[pq->flyers[0]->flyer_id] = 0;
        heapifyDown(pq, 0);
    }

    return max;
}

// Cancel a flyer's request - O(log n)
void cancelRequest(PriorityQueue* pq, int flyer_id) {
    int index = pq->id_to_index[flyer_id];
    if (index == -1) return; // Not found

    pq->flyers[index]->is_cancelled = true;
    pq->id_to_index[flyer_id] = -1;

    // Replace with last element and heapify
    if (index < pq->size - 1) {
        pq->flyers[index] = pq->flyers[pq->size - 1];
        pq->id_to_index[pq->flyers[index]->flyer_id] = index;
        pq->size--;

        // Heapify both ways to maintain heap property
        heapifyUp(pq, index);
        heapifyDown(pq, index);
    } else {
        pq->size--;
    }
}

// Find top k flyers - O(k log n)
void findTopKFlyers(PriorityQueue* pq, int k, Flyer** result, int* result_size) {
    *result_size = 0;

    while (*result_size < k && pq->size > 0) {
```

```
            Flyer* flyer = extractMax(pq);
            if (flyer != NULL && !flyer->is_cancelled) {
                result[*result_size] = flyer;
                (*result_size)++;
            }
        }
}

// Helper function to print flyer results
void printResults(Flyer** result, int result_size) {
        for (int i = 0; i < result_size; i++) {
            const char* status_str;
            switch (result[i]->status) {
                case SUPER: status_str = "Super"; break;
                case PLATINUM: status_str = "Platinum"; break;
                case GOLD: status_str = "Gold"; break;
                case SILVER: status_str = "Silver"; break;
            }
            printf("(%s, %d, %d, %s)\n", result[i]->name, result[i]->flyer_id,
                    result[i]->request_time, status_str);
        }
}

// Helper function to create a flyer
Flyer* createFlyerHelper(const char* name, int id, int time, Status status) {
        Flyer* f = (Flyer*)malloc(sizeof(Flyer));
        if (f == NULL) {
            printf("Memory allocation failed!\n");
            exit(1);
        }
        strncpy(f->name, name, MAX_NAME - 1);
        f->name[MAX_NAME - 1] = '\0';
        f->flyer_id = id;
        f->request_time = time;
        f->status = status;
        f->is_cancelled = false;
        return f;
}
```

```
// Function to run automated tests
void runAutomatedTests() {
    printf("==========================================\n");
    printf("Running Automated Test Cases\n");
    printf("==========================================\n\n");

    // ========== Test Case 1 ==========
    printf("=== Test Case 1 ===\n");
    printf("Input:\n5 2 1\n");
    printf("A 1 120 Platinum\nB 5 60 Super\nC 3 140 Super\n");
    printf("D 2 130 Gold\nE 4 150 Silver\n5\n\n");

    PriorityQueue* pq1 = createPriorityQueue();
    insertFlyer(pq1, createFlyerHelper("A", 1, 120, PLATINUM));
    insertFlyer(pq1, createFlyerHelper("B", 5, 60, SUPER));
    insertFlyer(pq1, createFlyerHelper("C", 3, 140, SUPER));
    insertFlyer(pq1, createFlyerHelper("D", 2, 130, GOLD));
    insertFlyer(pq1, createFlyerHelper("E", 4, 150, SILVER));
    cancelRequest(pq1, 5);

    Flyer* result1[100];
    int result_size1;
    findTopKFlyers(pq1, 2, result1, &result_size1);

    printf("Expected Output:\n(C, 3, 140, Super)\n(A, 1, 120, Platinum)\n\n");
    printf("Actual Output:\n");
    printResults(result1, result_size1);
    printf("\n");
    destroyPriorityQueue(pq1);

    // ========== Test Case 2 ==========
    printf("=== Test Case 2 ===\n");
    printf("Input:\n4 1 2\n");
    printf("A 1 30 Platinum\nB 2 60 Gold\nC 3 90 Silver\nD 4 120 Super\n");
    printf("2\n4\n\n");

    PriorityQueue* pq2 = createPriorityQueue();
```

```
insertFlyer(pq2, createFlyerHelper("A", 1, 30, PLATINUM));
insertFlyer(pq2, createFlyerHelper("B", 2, 60, GOLD));
insertFlyer(pq2, createFlyerHelper("C", 3, 90, SILVER));
insertFlyer(pq2, createFlyerHelper("D", 4, 120, SUPER));
cancelRequest(pq2, 2);
cancelRequest(pq2, 4);

Flyer* result2[100];
int result_size2;
findTopKFlyers(pq2, 1, result2, &result_size2);

printf("Expected Output:\n(A, 1, 30, Platinum)\n\n");
printf("Actual Output:\n");
printResults(result2, result_size2);
printf("\n");
destroyPriorityQueue(pq2);

// ========== Test Case 3 ==========
printf("=== Test Case 3 ===\n");
printf("Input:\n6 3 0\n");
printf("A 1 100 Gold\nB 2 50 Platinum\nC 3 200 Super\n");
printf("D 4 150 Super\nE 5 75 Platinum\nF 6 25 Gold\n\n");

PriorityQueue* pq3 = createPriorityQueue();
insertFlyer(pq3, createFlyerHelper("A", 1, 100, GOLD));
insertFlyer(pq3, createFlyerHelper("B", 2, 50, PLATINUM));
insertFlyer(pq3, createFlyerHelper("C", 3, 200, SUPER));
insertFlyer(pq3, createFlyerHelper("D", 4, 150, SUPER));
insertFlyer(pq3, createFlyerHelper("E", 5, 75, PLATINUM));
insertFlyer(pq3, createFlyerHelper("F", 6, 25, GOLD));

Flyer* result3[100];
int result_size3;
findTopKFlyers(pq3, 3, result3, &result_size3);

printf("Expected Output:\n(D, 4, 150, Super)\n(C, 3, 200, Super)\n(B, 2, 50,
Platinum)\n\n");
printf("Actual Output:\n");
```

```
    printResults(result3, result_size3);
    printf("\n");
    destroyPriorityQueue(pq3);

    // ========== Test Case 4 ==========
    printf("=== Test Case 4 (All Same Status - Time Priority) ===\n");
    printf("Input:\n5 3 1\n");
    printf("Alice 10 500 Gold\nBob 11 300 Gold\nCharlie 12 700 Gold\n");
    printf("David 13 200 Gold\nEve 14 600 Gold\n12\n\n");

    PriorityQueue* pq4 = createPriorityQueue();
    insertFlyer(pq4, createFlyerHelper("Alice", 10, 500, GOLD));
    insertFlyer(pq4, createFlyerHelper("Bob", 11, 300, GOLD));
    insertFlyer(pq4, createFlyerHelper("Charlie", 12, 700, GOLD));
    insertFlyer(pq4, createFlyerHelper("David", 13, 200, GOLD));
    insertFlyer(pq4, createFlyerHelper("Eve", 14, 600, GOLD));
    cancelRequest(pq4, 12);

    Flyer* result4[100];
    int result_size4;
    findTopKFlyers(pq4, 3, result4, &result_size4);

    printf("Expected Output:\n(David, 13, 200, Gold)\n(Bob, 11, 300, Gold)\n(Alice,
10, 500, Gold)\n\n");
    printf("Actual Output:\n");
    printResults(result4, result_size4);
    printf("\n");
    destroyPriorityQueue(pq4);

    printf("==========================================\n");
    printf("All Tests Completed!\n");
    printf("==========================================\n");
}

// Function to process user input
void processUserInput() {
    int n, k, c;
```

```
printf("==========================================\n");
printf("NUA Airline Upgrade System - Manual Input\n");
printf("==========================================\n\n");

printf("Enter n (number of flyers), k (seats available), c (cancellations):\n");
printf("Format: n k c\n> ");
scanf("%d %d %d", &n, &k, &c);

PriorityQueue* pq = createPriorityQueue();

// Read n flyers
printf("\nEnter %d flyers (one per line):\n", n);
printf("Format: Name FlyerID RequestTime Status\n");
printf("Status options: Super, Platinum, Gold, Silver\n");
for (int i = 0; i < n; i++) {
    Flyer* flyer = (Flyer*)malloc(sizeof(Flyer));
    char status_str[20];
    printf("Flyer %d> ", i + 1);
    scanf("%s %d %d %s", flyer->name, &flyer->flyer_id,
            &flyer->request_time, status_str);
    flyer->status = stringToStatus(status_str);
    flyer->is_cancelled = false;
    insertFlyer(pq, flyer);
}

// Read c cancellations
if (c > 0) {
    printf("\nEnter %d cancellation IDs (one per line):\n", c);
    for (int i = 0; i < c; i++) {
        int cancel_id;
        printf("Cancel %d> ", i + 1);
        scanf("%d", &cancel_id);
        cancelRequest(pq, cancel_id);
    }
}

// Find top k flyers
Flyer* result[MAX_FLYERS];
```

```
    int result_size;
    findTopKFlyers(pq, k, result, &result_size);

    // Output results
    printf("\n==========================================\n");
    printf("Output: Top %d Flyers Allowed to Board\n", k);
    printf("==========================================\n");
    for (int i = 0; i < result_size; i++) {
        const char* status_str;
        switch (result[i]->status) {
            case SUPER: status_str = "Super"; break;
            case PLATINUM: status_str = "Platinum"; break;
            case GOLD: status_str = "Gold"; break;
            case SILVER: status_str = "Silver"; break;
        }
        printf("(%s, %d, %d, %s)\n", result[i]->name, result[i]->flyer_id,
                result[i]->request_time, status_str);
        free(result[i]);
    }
    printf("\n");

    destroyPriorityQueue(pq);
}

// Main function with menu
int main(int argc, char* argv[]) {
    // Check if running in test mode
    if (argc > 1 && strcmp(argv[1], "test") == 0) {
        runAutomatedTests();
        return 0;
    }

    // Interactive menu
    printf("==========================================\n");
    printf("NUA Airline Upgrade System\n");
    printf("==========================================\n\n");
    printf("Select mode:\n");
    printf("1. Manual Input (Enter your own test case)\n");
```

```
    printf("2. Run Automated Tests\n");
    printf("3. Exit\n\n");
    printf("Choice> ");

    int choice;
    scanf("%d", &choice);
    printf("\n");

    switch (choice) {
        case 1:
            processUserInput();
            break;
        case 2:
            runAutomatedTests();
            break;
        case 3:
            printf("Exiting program. Goodbye!\n");
            break;
        default:
            printf("Invalid choice. Exiting.\n");
    }

    return 0;
}
```

/*

Test Case 1:

Input:

5 2 1

A 1 120 Platinum

B 5 60 Super

C 3 140 Super

D 2 130 Gold

E 4 150 Silver

5

Expected Output:

(C, 3, 140, Super)

(A, 1, 120, Platinum)

Test Case 2:
Input:
4 1 2
A 1 30 Platinum
B 2 60 Gold
C 3 90 Silver
D 4 120 Super
2
4

Expected Output:
(A, 1, 30, Platinum)

Test Case 3:
Input:
6 3 0
A 1 100 Gold
B 2 50 Platinum
C 3 200 Super
D 4 150 Super
E 5 75 Platinum
F 6 25 Gold

Expected Output:
(D, 4, 150, Super)
(C, 3, 200, Super)
(B, 2, 50, Platinum)

Test Case 4:
Input:
5 3 1
Alice 10 500 Gold
Bob 11 300 Gold
Charlie 12 700 Gold
David 13 200 Gold
Eve 14 600 Gold

12

Expected Output:
(David, 13, 200, Gold)
(Bob, 11, 300, Gold)
(Alice, 10, 500, Gold)
****/***